



Security Assessment

Final Report



Polymarket Perpetuals

Contract v1

April 2026

Prepared for Polymarket

Table of Contents

Project Summary	3
Project Scope	3
Project Overview	3
Protocol Overview	3
Assessment Methodology	5
Threat and Security Overview	6
Findings Summary	8
Severity Matrix	8
Detailed Findings	9
Medium Severity Issues	10
M-01: Carried-forward balances may be withdrawable twice if a new root does not include recently processed withdrawals	10
Low Severity Issues	12
L-01: Deposit minimum check uses requested amount instead of actual received amount	12
Informational Severity Issues	14
I-01: Token donation can break the invariant sum of all leaf balances equals the contract's on-chain token balance	14
I-02: Fee accounting relies on offchain state transitions and is not fully observable onchain	16
I-03: Withdrawal fee is not enforced onchain and relies on offchain validation	17
I-04: Cross-margin debt can be selectively bypassed during isolated token withdrawal	19
I-05: ReentrancyGuardTransient makes deposits and withdrawals unusable on chains without EIP-1153 support	20
I-06: StateRootCommitted event does not include epoch	21
Disclaimer	22
About Certora	22

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Polymarket Perpetuals Contract v1	https://github.com/Polymarket/perpetuals-contract	df69be7 (initial) ab083b3 (latest)	Solidity

Project Overview

This document describes the security review of **Polymarket Perpetuals Contract v1**. The work was undertaken from **April 27th, 2026** to **April 28th, 2026**.

The following contract list is included in our scope:

```
src/v1/*
```

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

This system is essentially an **onchain vault + settlement layer** that works alongside an offchain trading engine. All the heavy lifting like order matching, balance updates, funding, and fee accounting happens offchain, while this contract holds user funds and periodically “anchors” the latest state onchain using a Merkle root.

Each **stateRoot** represents a snapshot of all user balances (**account, token, balance**) at a point in time. The full data used to build this root is published offchain.

On a high level, the flow is:

- Users **deposit** tokens into the contract.

- The offchain engine updates balances based on trades, deposits, etc.
- The operator periodically commits a new **state root**.
- Users sign withdrawal messages, and **keepers** execute them onchain using a proof + signature.

The system uses a simple role setup:

- **Owner:** manages permissions and upgrades
- **Operator:** runs the offchain engine and commits roots
- **Keeper:** executes withdrawals
- **User:** deposits funds and signs withdrawals

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

The system follows a **hybrid trust model**, where onchain logic enforces signature verification and Merkle inclusion, but overall correctness depends heavily on the offchain engine.

Trust assumptions

- The **operator (offchain engine)** is trusted to construct correct state roots, maintain solvency, and ensure each user's balance is accurately carried forward across roots.
- The operator must also ensure **leaf uniqueness** and correct inclusion of all state transitions (deposits, trades, fees, withdrawals).
- **Keepers** are trusted for liveness, i.e., to actually process user withdrawals in a timely manner.
- The owner is fully trusted and must be a well-managed multisig. The compromised owner can:
 1. Upgrade the contract (via `_authorizeUpgrade`) to a malicious implementation that can drain all funds. There is no timelock, so this can happen in a single block.
 2. Set operators and keepers (`setKeeper`) to be malicious so the operator can commit a fake `stateRoot` and the keeper can process fake withdrawal requests
- Ownership transfer is a 2-step process but has no delay, so it does not protect against a compromised owner key.

Onchain guarantees

- Funds are held in the contract and can only be withdrawn with:
 - a valid **EIP-712 signature** from the user, and
 - a valid **Merkle proof** against the latest committed state root
- Replay of withdrawal signatures is prevented via `usedDigests`.
- Partial withdrawals under the same root are tracked via `withdrawn[root][token][account]`.

Key risks / considerations

- **State correctness is offchain-dependent:** the contract does not verify how roots are built, so incorrect or stale roots can lead to incorrect balances being enforced onchain.
- **Liveness dependency:** users cannot withdraw directly; they rely on keepers to process signed withdrawals.
- **Fee enforcement:** protocol fees are not enforced onchain and rely on offchain validation and accounting.
- **Owner key compromise means total fund loss:** with no timelock on upgrades, a compromised owner can drain all funds in a single transaction via `upgradeToAndCall` (upgrade + drain in one call). The owner key should be a multisig, ideally protected by a Timelock, so users can respond to a malicious upgrade.
- **No emergency pause:** the contract has neither a pause function nor a withdrawal cap. If a fraudulent state root is committed, there is no on-chain way to stop withdrawals before keepers process them.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	1	1	1
Low	1	1	1
Informational	6	6	4
Total	8	8	6

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
M-01	Carried-forward balances may be withdrawable twice if a new root does not include recently processed withdrawals	Medium	Fixed
L-01	Deposit minimum check uses requested amount instead of actual received amount	Low	Fixed
I-01	Token donation can break the invariant sum of all leaf balances equals the contract's on-chain token balance	Informational	Fixed
I-02	Fee accounting relies on offchain state transitions and is not fully observable onchain	Informational	Acknowledged
I-03	Withdrawal fee is not enforced onchain and relies on offchain validation	Informational	Fixed
I-04	Cross-margin debt can be selectively bypassed during isolated token withdrawal	Informational	Fixed
I-05	ReentrancyGuardTransient makes deposits and withdrawals unusable on chains without EIP-1153 support	Informational	Acknowledged
I-06	StateRootCommitted event does not include epoch	Informational	Fixed

Medium Severity Issues

M-01: Carried-forward balances may be withdrawable twice if a new root does not include recently processed withdrawals

Severity: Medium	Impact: High	Likelihood: Low
Files: src/v1/ExchangeV1.sol	Status: Fixed	

Description:

There can exist a scenario where a new state root is built before a withdrawal against the previous root is processed on chain. Since withdrawal accounting is scoped per root via `withdrawn[root][token][account]`, the contract does not itself ensure that amounts withdrawn under an older root are deducted from the next root.

This depends on the keeper/operator flow, which is offchain and currently a black box.

Consider the following scenario ->

1. Root R1 shows Alice with 100 withdrawable.
2. Alice deposits 50, so the next intended cumulative balance may be 150.
3. Before R2 is committed via `commitStateRoot()`, Alice signs a withdrawal for 100 against R1.
4. The keeper processes the R1 withdrawal, so Alice receives 100.
5. R2 is then committed showing Alice with 150, because it was built without accounting for the recent R1 withdrawal.
6. Alice signs another withdrawal and withdraws 150 from R2.

Alice receives 250 total, while her correct cumulative balance should only be 150.

Recommendations:

The keepers should pause withdrawal processing when the operator commits a new state root. And the operator should ensure that a new state root is committed only after all withdrawals have been processed and confirmed at sufficient depth (to avoid reorg-related attacks), and that those withdrawals are accounted for when building the state root.

Customer Response:

Mitigated offchain. Operator keeps a live state root and validates incoming withdrawals against it, debiting every approved withdrawal (offchain or onchain) before committing the next root documented in README.md under operator guarantee #3 following this commit [285724a](#).

Fix Review:

Mitigation looks good.

Low Severity Issues

L-01: Deposit minimum check uses requested amount instead of actual received amount

Severity: Low	Impact: Low	Likelihood: Low
Files: src/v1/ExchangeV1.sol	Status:	

Description:

The contract now calculates and emits the actual received amount:

```
uint256 balanceBefore = IERC20(token).balanceOf(address(this));
IERC20(token).safeTransferFrom(msg.sender, address(this), amount);
uint256 received = IERC20(token).balanceOf(address(this)) - balanceBefore;

emit Deposit(msg.sender, token, received, to);
```

However, the minimum deposit check is still performed against the requested **amount**:

```
uint256 minimum = supportedAssets[token];
if (minimum == 0) revert AssetNotSupported();
if (amount < minimum) revert DepositBelowMinimum();
```

So if a token transfers less than requested, the deposit can pass the minimum check while the actual credited **received** amount is below the configured minimum.

Recommendations:

Consider checking the minimum against **received** as well:

```
if (received < minimum) revert DepositBelowMinimum();
```

Customer Response:

Fixed in [522b6ee](#).



Fix Review:

Fix confirmed.

Informational Severity Issues

I-01: Token donation can break the invariant sum of all leaf balances equals the contract's on-chain token balance

Files:

src/v1/ExchangeV1.sol
README.md

Description:

The [README.md](#) states that the operator guarantees solvency in the sense that the sum of all Merkle leaf balances for a token equals the contract's on-chain balance of that token.

```
### operator guarantees
```

The operator guarantees two system-wide invariants:

1. ***solvency***: the sum of all leaf balances equals the contract's on-chain token balance. Every user balance is fully backed, so a simultaneous mass withdrawal through any channel, including the escape hatch, leaves every user whole.

Anyone can increase the ERC-20 balance of the [ExchangeV1](#) contract by donating some token to the contract, without calling [deposit](#) or [depositWithPermit](#) functions. That path does not emit the [Deposit](#) event and does not trigger a matching leaf update. So this breaks the invariant that the sum of all leaf balances equals the contract's on-chain token balance.

If someone calculates the sum of all the leaf balances from the transition data published by the project, then they can find the mismatch. It might cause some doubts regarding the operator and the keeper's honesty. However, this does not break solvency because the contract token balance is sufficient to support simultaneous mass withdrawals by users.

Recommendations:

Consider one of the following options:

- implementing logic in the off-chain engine to account for any donation amount as fee to the fee recipient
- clarify the Readme to acknowledge this risk
- implement internal balance tracking for all the supported tokens.

Customer Response:

README change. Will relax the invariant from $=$ to $\text{sum}(\text{leaves}) \leq \text{contract balance}$ and note the operator may credit donations to the fee recipient via offchain transition [a35f33d](#).

Fix Review:

Fix confirmed.

I-02: Fee accounting relies on offchain state transitions and is not fully observable onchain

Files:

src/v1/ExchangeV1.sol

Description:

The contract deducts a fee during withdrawals by transferring only **amount - fee** to the recipient, leaving the fee portion in the contract. While withdrawal fees are visible as they are passed into **processWithdrawal()**, broader fee components (e.g., trading fees) are not directly observable onchain and are expected to be incorporated into the offchain state. This means correct fee attribution (e.g., to a treasury account) depends entirely on the accuracy and transparency of the published state transition data.

Recommendations:

Consider adding explicit onchain accounting else consider acknowledging the issue.

Customer Response:

Acknowledged, documented in README.md under fees.

I-03: Withdrawal fee is not enforced onchain and relies on offchain validation

Files:

src/v1/ExchangeV1.sol

Description:

The withdrawal flow accepts a user-signed fee parameter and only enforces that `fee < amount` -->

```
function processWithdrawal(
    address account,
    address token,
    uint256 amount,
    uint256 fee,
    address to,
    uint64 salt,
    uint64 timestamp,
    bytes calldata signature,
    int256 balance,
    bytes32[] calldata proof
) external virtual onlyKeeper nonReentrant {
    if (amount == 0) revert ZeroAmount();
    if (fee >= amount) revert FeeTooHigh();
```

There is no onchain validation of minimum fee, fee rate, or protocol defined fee logic. As a result, a withdrawal signed with `fee = 0` is considered valid and will transfer the full amount to the recipient if processed by a keeper.

This behavior assumes that fee correctness is enforced entirely by the offchain system (e.g., order generation, keeper validation, or backend checks). The contract itself does not impose any constraints on the fee beyond basic bounds.

Recommendations:

Document that fee enforcement is handled offchain and that keepers are expected to validate fee correctness before processing withdrawals.



Customer Response:

README fixed in [1128961](#).

Fix Review:

Fix confirmed.

I-04: Cross-margin debt can be selectively bypassed during isolated token withdrawal

Files:

src/v1/ExchangeV1.sol

Description:

The Merkle tree intentionally stores account balances as `int256`, explicitly accommodating negative balances (debt) for a cross-margin architecture. However, the `processWithdrawal` function processes user withdrawals strictly on an isolated, per-token basis. It only validates the Merkle proof for the requested token and verifies that its standalone balance is positive. A malicious user can exploit this to selectively withdraw all of their positive collateral while abandoning negative debt balances in other tokens, leading to systemic insolvency.

Recommendations:

The off-chain engine logic of the keeper should have a health check for all token balances of the account before accepting and processing the withdrawal requests

Customer Response:

Documented in README.md under operator guarantee #3 following this commit [285724a](#).

Fix Review:

Mitigation looks good.

I-05: ReentrancyGuardTransient makes deposits and withdrawals unusable on chains without EIP-1153 support

Files:

src/v1/ExchangeV1.sol

Description:

The contract utilizes OpenZeppelin's `ReentrancyGuardTransient`, which depends on EIP-1153 transient storage opcodes (TSTORE/TLOAD). Deploying this contract on an EVM-compatible chain lacking EIP-1153 support will cause critical functions like `deposit`, `depositWithPermit` and `processWithdrawal` to fail. Polymarket contracts are currently deployed only on Polygon Mainnet, which has supported TSTORE/TLOAD since the Napoli hardfork (activated on Polygon mainnet in 2024), so this is not an issue for the current deployment target. Most major EVM mainnets (Ethereum, Polygon, Arbitrum, Optimism, Base, etc.) now support EIP-1153. However, this should still be kept in mind: any future deployment to a new chain (especially L2S, sidechains, or alternative EVM chains) must verify EIP-1153 support before deployment to avoid breaking deposits and withdrawals.

Recommendations:

Confirm that all intended deployment networks fully support EIP-1153 prior to deployment. If multi-chain compatibility is required, rely on the standard storage-based `nonReentrant`.

Customer Response:

Acknowledged, will keep it as is for now.

I-06: StateRootCommitted event does not include epoch

Files:

src/v1/ExchangeV1.sol

Description:

Withdrawals are now tracked per epoch:

```
mapping(uint256 epoch => mapping(address token => mapping(address account => uint256 amount)))) public withdrawn;
```

and each root commit increments epoch:

```
stateRoot = _stateRoot;
unchecked {
    ++epoch;
}

emit StateRootCommitted(_stateRoot, _hash);
```

However, the emitted event only includes `_stateRoot` and `_hash`, not the new epoch. This makes it harder for offchain systems, indexers, and users to clearly map each committed root to its withdrawal accounting period.

Recommendations:

Include `epoch` in the `StateRootCommitted` event.

Customer Response:

Fixed in [ab083b3](#).

Fix Review:

Fix confirmed.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.